



Introdução a Técnicas de Programação

Strings

Prof. André Campos
DIMAp/UFRN



O que são strings?

Representações de textos = **sequência de caracteres**

“caractere” $\Rightarrow$ ‘c’, ‘a’, ‘r’, ‘a’, ‘c’, ‘t’, ‘e’, ‘r’, ‘e’

0	1	2	3	4	5	6	7	8	9
c	a	r	a	c	t	e	r	e	

“ ” - representa uma string

‘ ’ - representa um caractere

Espaço alocado $\neq$ espaço utilizado

Em alguns problemas, definimos um vetor com um grande número de espaços, mas só utilizamos parte dele.

Ex: Uma fila, que inserimos no final

Nesses casos, precisamos saber
“onde os dados terminam”

Para não ter uma 2ª variável guardando o índice de “fim de dados”, as strings usam um “caractere especial”.

Caractere Nulo = `'\0'`

Vetor vazio					
Inserir 1	1				
Inserir 4	1	4			
Inserir 2	1	4	2		
Inserir 7	1	4	2	7	

Exemplos de strings

Todas strings foram armazenadas em um vetor de 17 caracteres

	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16
Mama mia	M	a	m	a		m	i	a	\0								
La vita è bella	L	a		v	i	t	a		è		b	e	l	l	a	\0	
Volare	V	o	l	a	r	e	\0										
La solitudine	L	a		s	o	l	i	t	u	d	i	n	e	\0			
Sapore di sale	S	a	p	o	r	e		d	i		s	a	l	e	\0		



Strings em C++

Pode-se trabalhar com

- Sequências do tipo **char** (C-string)
- Classe pré-definida (usa as sequências de char do C e “esconde os detalhes”)

```
char name[50];  
cout << "Digite seu nome: ";  
cin >> name;  
cout << name << endl;
```

```
string name;  
cout << "Digite seu nome: ";  
cin >> name;  
cout << name << endl;
```



C strings - sequências de caracteres

Exatamente como definimos um vetor de char

```
char string[30];
```

Porém, há outra forma de inicializar

```
char greetings[] = "Olá pessoal!";  
char bye[20] = "Até mais!";
```

Quando definir explicitamente o tamanho, lembre-se de um espaço para o caractere nulo `'\0'`. Então, coloque no mínimo o tamanho do texto + 1.

Lendo sequências de caracteres

Entendendo um buffer

```
$ Digite o nome do filme:  
Mama mia
```

buffer

M	a	m	a		m	i	a	\n								
---	---	---	---	--	---	---	---	----	--	--	--	--	--	--	--	--

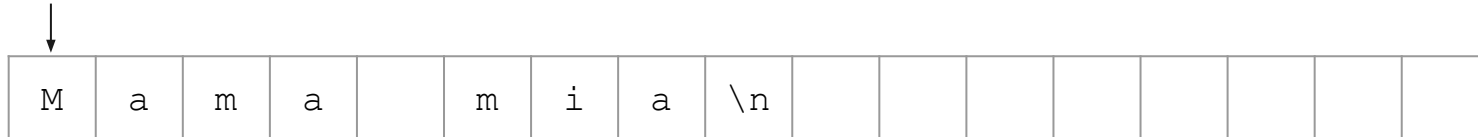
Por questões de eficiência, os dados são enviados inicialmente para uma área de armazenamento (buffer) e depois lidos

A leitura da string será feita até um caracteres “branco” (espaço, tab, salto de linha...)

Lendo sequências de caracteres

Leitura no buffer

início do
buffer

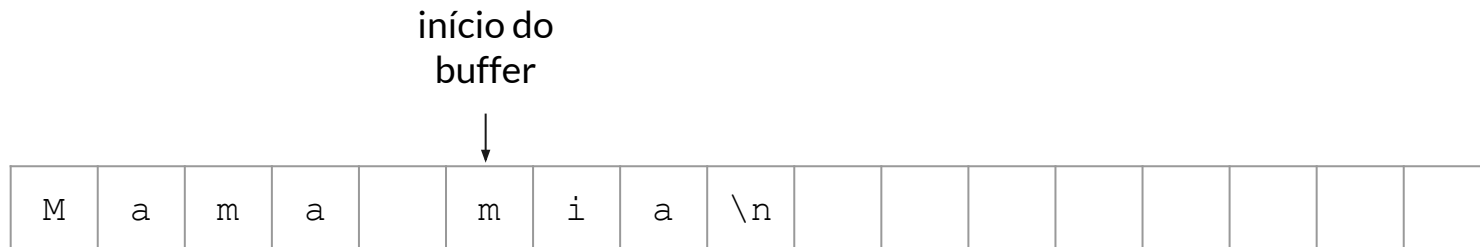


```
char filme[50];  
cout << "Digite o nome do filme:\n";  
cin >> filme;  
cout << filme << endl;
```

```
$ Digite o nome do filme:  
Mama mia
```


Lendo sequências de caracteres

Leitura no buffer



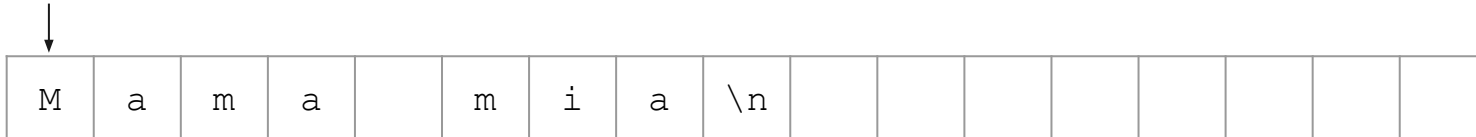
```
char filme[50];  
cout << "Digite o nome do filme:\n";  
cin >> filme;  
cout << filme << endl;
```

```
$ Digite o nome do filme:  
Mama mia  
Mama
```

Lendo sequências de caracteres

E se o caractere “branco” estiver além da capacidade da sequência?

início do
buffer



```
char filme[2];  
cout << "Digite o nome do filme:\n";  
cin >> filme;  
cout << filme << endl;
```

```
$ Digite o nome do filme:  
Mama mia  
*** stack smashing detected ***: terminated
```

** exceto se usar o C++20*

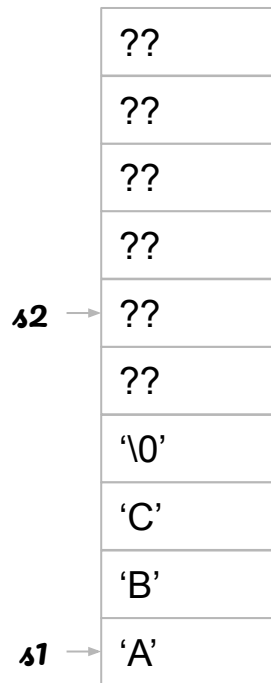
Operações comuns em strings

Usando um array de chars, não se pode simplesmente tratá-lo como se fosse uma variável primitiva (int, bool...)

Ex: atribuição

```
char s1[5] = "ABC";  
char s2[5];  
s2 = s1; ❌
```

Memória





Rotinas comuns de sequências de caracteres

Como é um array, não se pode simplesmente tratar como variáveis primitivas

Ex: leitura e entrada de dados

Algumas rotinas para manipular sequências de caracteres em `<cstring>`

- `strlen(str)` → retorna o tamanho da string (não o espaço alocado!)
- `strcpy(str1, str2)` → copia uma string para um vetor de char
- `strcmp(str1, str2)` → compara duas strings (ordem lexicográfica)
- `strcat(str1, str2)` → junta duas strings
- `strstr(str1, str2)` → verifica se uma string faz parte de outra



strlen()

Retorna quantos caracteres uma string possui.

Percorre caractere a caractere até achar o “fim de string” (\0)

Similar a...

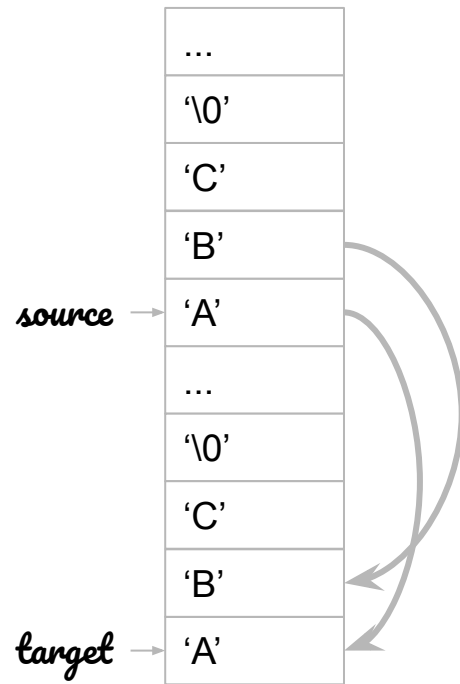
```
int strlen(char string[]){  
    int i;  
    for (i = 0; string[i] != '\0'; i++);  
    return i;  
}
```

strcpy()

Similar a...

```
void strcpy(char target[], char source[]){  
    int i;  
    for (i = 0; source[i] != '\0'; i++) {  
        target[i] = source[i];  
    }  
    target[i] = '\0';  
}
```

Memória





Outras funções

`strcmp(str1, str2)`

Compara de forma lexicográfica duas strings (ex: “a” < “b”) e retorna:

- -1 se `str1 > str2`
- 0 se `str1 = str2`
- 1 se `str1 < str2`

`strcat(str1, str2)`

Concatena duas strings. Adiciona `str2` no final de `str1` e retorna `str1`.

`strstr(str1, str2)`

Retorna o endereço da 1ª ocorrência de `str2` em `str1` ou NULL se não houver `str2` dentro de `str1`.



Classe string

Encapsula o array de caracteres, de modo a:

- Simplifica as operações com strings
- Tornar tamanho do array dinâmico

O acesso aos caracteres continuam como um array de char

```
string name, surname, fullname;
cin >> name >> surname;
fullname = name + " " + surname;
cout << fullname;
if (name == surname) {
    cout << "Você digitou duas vezes\n";
}
```




Operações do tipo string

Há várias operações que uma variável do tipo string pode realizar:

- `length()` ou `size()`: retorna o número de caracteres
- `append()` ou o operador `+`: concatena duas strings
- `compare()` ou o operador `==`: compara a igualdade de duas strings
- `substr()`: extrai uma substring da original
- `find()`: retorna a posição da primeira ocorrência
- `replace()`: substitui uma parte da string
- `c_str()`: transforma em array de caracteres (C-string)
- outras...



Lendo uma linha inteira

Uso do `getline()`

```
string name;  
getline(cin, name);  
cout << name;
```



Percorrendo todos os caracteres de uma string

Laço similar aos arrays

```
string str;  
cin >> str;  
  
for(int i = 0; i < str.length(); i++) {  
    cout << str[i] << " ";  
}  
cout << endl;
```

ou

```
string str;  
cin >> str;  
  
for(char c: str) {  
    cout << c << " ";  
}  
cout << endl;
```



Práticas

1. Use uma string como array de char e implemente sua versão de:
 - a. `strlen()`
 - b. `strcat()`
 - c. `strstr()`

Obs: Use outro nome para o compilador não reclamar.

2. Implemente uma função que recebe uma string e transforma seus caracteres em letras maiúsculas, seguindo a assinatura abaixo:

```
void upper(string str);
```

3. Dadas duas strings `str1` e `str2`, implemente uma função que retorne o número de ocorrências de `str2` em `str1`.

Problema

José quer impressionar Maria escrevendo um poema para ela, mas criar versos com rima não é seu forte. Sempre que escreve algo, fica procurando um termo para rimar e não consegue. Ele, então, teve a ideia de pedir tua ajuda para escrever um programa que listasse todas as palavras do dicionário que poderiam rimar com a palavra que ele quisesse.

Para começar, você pretende apenas testar se uma palavra rima com outra.





Tarefa 1

Escreva um programa que lê duas strings P e T e verifica se P termina com T. As strings podem estar em maiúscula ou minúscula.

Entrada

A primeira linha contém a string P e a segunda linha contém a string T.

Saída

O programa deve imprimir “rima” se P termina em T ou “não rima” caso contrário.

Exemplos de entrada e saída

Entrada	Saída
cabrocha acha	não rima

Entrada	Saída
Beleza eza	rima

Entrada	Saída
Beleza pureza	não rima

Problema

Maria, impressionada com o poema de José, enviou-lhe uma mensagem-resposta: *“Ainda dá olhos rápidos e imaginação”*.

José não entendeu nada!!! Mas, você, esperto, viu que a mensagem estava codificada. Maria quis dizer “Adorei”.

Para ajudar José a decifrar o que Maria responde, ele te pediu para escrever um programa que decodificar as mensagens dela.





Tarefa

Escreva um programa que lê uma frase e imprime as primeiras letras de cada palavra.

Entrada

Consiste em uma linha contendo um texto com maiúsculas ou minúsculas (sem acentos ou caracteres especiais). Pode haver mais de um espaço e pontuações entre as palavras do texto. O texto pode também iniciar ou terminar em espaços, ou mesmo conter somente espaços.

Saída

O programa deve imprimir as letras iniciais de cada palavra do texto de entrada, todas em letras minúsculas.

Exemplos de entrada e saída

Entrada	Saída
Ainda da olhos rapido e imaginacao	adorei

Entrada	Saída
Foi ultimo instalado	fui

Entrada	Saída
Beleza! Adorei cair agora no altar	bacana